

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: DESIGN AND ANALYSIS OF ALGORITHMS

COURSE CODE: CSA06COURSE OUTCOME COVERED

NAME: A SHIVA SHANKAR REDDY

REG NO:192425174

CO 4: Solve optimization problems using dynamic programming and greedy strategies.

(BL4, BL5)

Annexure A

Industry Problem Solving Task

Q1. Knapsack – Industry Resource Allocation

A cloud service provider must allocate limited server memory (100 GB) among multiple client applications, each having different memory requirements and revenue values. Analyze the problem and explain how the 0/1 Knapsack algorithm can be applied to maximize total revenue. Design a feasible solution using dynamic programming and justify the selection of this approach. Discuss how overlapping subproblems and optimal substructure are utilized. Suggest suitable tools or platforms for implementation. Analyze the performance in terms of time and space complexity and interpret the results in an industry context.

1. Problem Statement

A cloud service provider has a limited server memory capacity of 100 GB that must be allocated among multiple client applications. Each client application requires a fixed amount of memory (weight) and generates a fixed amount of revenue (value) if it is hosted on the server. Since total memory is limited, and each application must be either fully hosted or not hosted at all (it cannot be partially allocated memory), the provider must choose a subset of applications that maximizes total revenue while keeping total memory usage within the 100 GB limit.

This scenario is a direct real-world instance of the classical 0/1 Knapsack Problem, where the server memory capacity corresponds to the knapsack capacity, each application corresponds to an item, the memory requirement corresponds to item weight, and the revenue generated corresponds to item value.

2. Mapping the Problem to 0/1 Knapsack

Knapsack Element	Cloud Resource Allocation Equivalent
Knapsack capacity (W)	Total available server memory = 100 GB
Items	Client applications requesting memory
Weight of item i	Memory required by application i (in GB)
Value of item i	Revenue generated by hosting application i
0/1 constraint	An application is either fully hosted (1) or not hosted (0); partial hosting is not allowed
Objective	Maximize total revenue subject to total memory used $\leq$ 100 GB

Sample dataset used for implementation and testing:

Application	Memory Required (GB)	Revenue Generated (₹)
App 1	10	60
App 2	20	100
App 3	30	120
App 4	40	150
App 5	50	200

3. Justification for Choosing Dynamic Programming

A greedy strategy, such as selecting applications in order of highest revenue-per-GB ratio, does not guarantee an optimal solution for the 0/1 Knapsack Problem. Greedy selection works correctly only for the Fractional Knapsack variant, where items can be split. Because server memory in this scenario must be allocated to an application in full or not at all, a greedy approach can lead to a suboptimal allocation by locking in a locally attractive application that prevents a better combination of applications from fitting into the remaining memory. Dynamic Programming (DP) evaluates all feasible combinations systematically and is therefore required to guarantee the globally optimal allocation.

3.1 Optimal Substructure

The maximum revenue achievable using the first i applications with a memory budget of w GB depends only on optimal solutions to smaller subproblems: either application i is excluded, in which case the optimal solution is the same as the optimal solution for the first $i-1$ applications with the same budget w ; or application i is included, in which case the optimal solution equals its own revenue plus the optimal solution for the first $i-1$ applications with the reduced budget (w minus the memory required by application i). Because the overall optimum is constructed from optimal solutions to these smaller subproblems, the problem exhibits optimal substructure.

3.2 Overlapping Subproblems

When the problem is solved using plain recursion, the same subproblem (a specific number of applications considered together with a specific remaining memory capacity) is encountered repeatedly across different branches of the recursion tree. Dynamic Programming exploits this by storing each subproblem's result in a table (memoization in the top-down approach, or a table built iteratively in the bottom-up/tabulation approach), so that every distinct subproblem is computed exactly once and reused whenever it recurs. This eliminates redundant computation and reduces the running time from exponential to polynomial.

4. DP Formulation and Recurrence Relation

Let $dp[i][w]$ denote the maximum revenue achievable using the first i applications with a memory budget of w GB.

```
dp[0][w] = 0                                for all w    (no applications
considered)
dp[i][w] = dp[i-1][w]                        if weight[i] > w
dp[i][w] = max( dp[i-1][w],                  value[i] + dp[i-1][w - weight[i]] )    if weight[i] <= w
```

The final answer (maximum total revenue) is stored in $dp[n][W]$, where n is the number of applications and $W = 100$ GB. The set of applications actually hosted is recovered by backtracking through the table.

5. Dynamic Programming Solution – Implementation (Python)

The program below builds the DP table bottom-up (tabulation) and backtracks through it to identify which applications should be hosted:

```
def knapsack_allocation(capacity, weights, values, names):
    n = len(weights)
    # dp[i][w] = maximum revenue using the first i applications
    #             with a memory budget of w GB
    dp = [[0 for _ in range(capacity + 1)] for _ in range(n + 1)]

    for i in range(1, n + 1):
        for w in range(capacity + 1):
            if weights[i - 1] <= w:
                dp[i][w] = max(dp[i - 1][w],
                               values[i - 1] + dp[i - 1][w - weights[i - 1]])
            else:
                dp[i][w] = dp[i - 1][w]

    # Backtrack through the table to find the applications selected
    selected, w = [], capacity
    for i in range(n, 0, -1):
        if dp[i][w] != dp[i - 1][w]:
            selected.append(names[i - 1])
            w -= weights[i - 1]

    return dp[n][capacity], selected[::-1]

if __name__ == '__main__':
    memory_capacity = 100 # total server memory (GB)
    app_names = ['App1', 'App2', 'App3', 'App4', 'App5']
    app_weights = [10, 20, 30, 40, 50] # memory requirement (GB)
    app_values = [60, 100, 120, 150, 200] # revenue generated

    max_revenue, apps_hosted = knapsack_allocation(
        memory_capacity, app_weights, app_values, app_names)

    print(f'Maximum achievable revenue : {max_revenue}')
    print(f'Applications hosted : {apps_hosted}')
```

5.1 Program Output

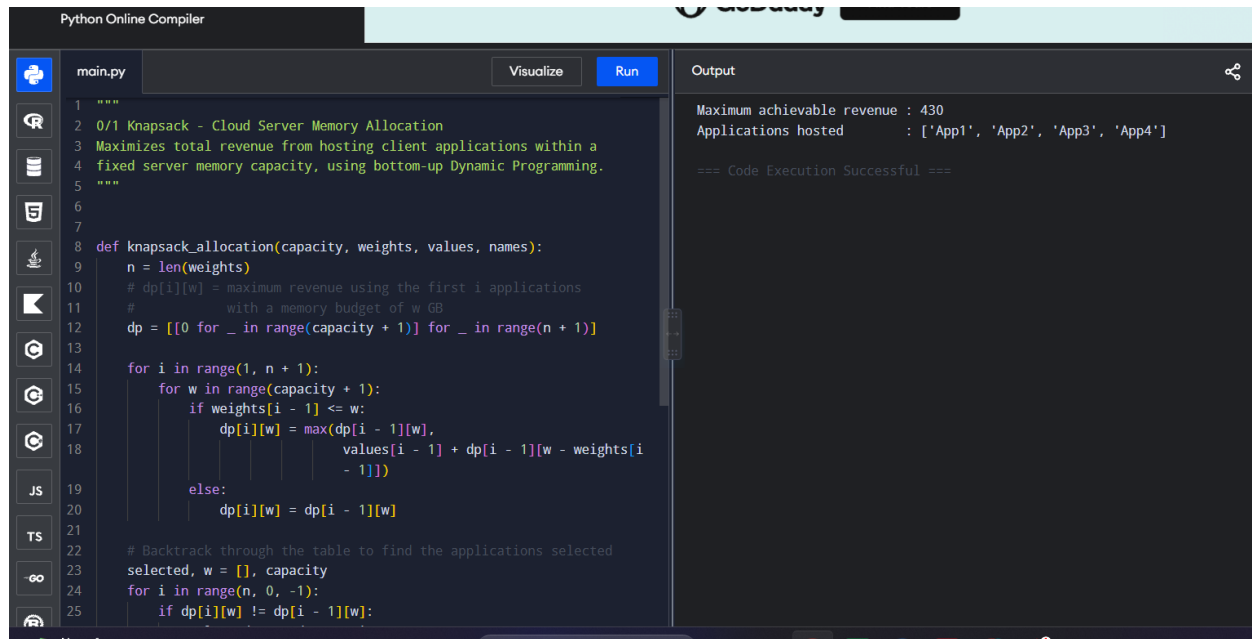
Executing the program on the sample dataset (capacity = 100 GB) produces:

Maximum achievable revenue : 430

Applications hosted : ['App1', 'App2', 'App3', 'App4']

Interpretation: Hosting App1, App2, App3 and App4 consumes exactly $10 + 20 + 30 + 40 = 100$ GB of memory (the full available capacity) and generates the maximum possible revenue of ₹430. App5, despite offering the single highest individual revenue (₹200), is excluded because including it would prevent enough of the other applications from fitting, resulting in lower total revenue – illustrating precisely why a greedy ‘highest value first’ rule can fail where DP succeeds.

OUTPUT:



The screenshot displays a Python Online Compiler interface. The left pane shows a file named 'main.py' with the following code:

```
1 """
2 0/1 Knapsack - Cloud Server Memory Allocation
3 Maximizes total revenue from hosting client applications within a
4 fixed server memory capacity, using bottom-up Dynamic Programming.
5 """
6
7
8 def knapsack_allocation(capacity, weights, values, names):
9     n = len(weights)
10    # dp[i][w] = maximum revenue using the first i applications
11    # with a memory budget of w GB
12    dp = [[0 for _ in range(capacity + 1)] for _ in range(n + 1)]
13
14    for i in range(1, n + 1):
15        for w in range(capacity + 1):
16            if weights[i - 1] <= w:
17                dp[i][w] = max(dp[i - 1][w],
18                             values[i - 1] + dp[i - 1][w - weights[i - 1]])
19            else:
20                dp[i][w] = dp[i - 1][w]
21
22    # Backtrack through the table to find the applications selected
23    selected, w = [], capacity
24    for i in range(n, 0, -1):
25        if dp[i][w] != dp[i - 1][w]:
```

The right pane shows the output of the code execution:

```
Maximum achievable revenue : 430
Applications hosted       : ['App1', 'App2', 'App3', 'App4']

=== Code Execution Successful ===
```

6. Suggested Tools and Platforms for Implementation

- Programming language: Python 3 (or Java/C++) for implementing and testing the DP algorithm.
- IDE / Editor: Visual Studio Code or PyCharm for development and debugging.
- Version control & submission: Git and GitHub, for hosting the Source_Code and Report as required by the submission instructions.
- Execution/testing: Python's built-in unittest module or manual test cases for verifying correctness against multiple datasets.
- Visualization (optional): Matplotlib, to plot the DP table growth or revenue vs. capacity, aiding the analysis section.
- Cloud context tools (optional, for realism): AWS/Azure/GCP resource dashboards can be referenced to relate memory allocation values to real server specifications.

7. Performance Analysis – Time and Space Complexity

Approach	Time Complexity	Space Complexity
Naive Recursion	$O(2^n)$	$O(n)$ (recursion stack)
DP – Top-Down (Memoization)	$O(n \times W)$	$O(n \times W) + O(n)$ stack
DP – Bottom-Up (Tabulation)	$O(n \times W)$	$O(n \times W)$
DP – Space-Optimized (1-D array)	$O(n \times W)$	$O(W)$

Here n is the number of client applications and W is the total server memory capacity (100 GB). The tabulation approach used in the implementation above runs in $O(n \times W)$ time and uses $O(n \times W)$ space to build the full table, which also allows backtracking to identify the exact set of hosted applications. If only the maximum revenue value is required (and not the list of applications), the space complexity can be reduced to $O(W)$ by using a single 1-D array and iterating the capacity dimension from high to low, since each row of the DP table only depends on the row immediately above it.

In an industry context, W (server memory in GB) is typically a fixed, moderate-sized value (e.g., 100 GB in this problem), and n (number of client applications competing for that memory) is also bounded. This makes the $O(n \times W)$ pseudo-polynomial time complexity of the DP solution entirely practical for real-time or near-real-time resource allocation decisions, unlike the exponential $O(2^n)$ naive recursive approach, which becomes infeasible even for a moderate number of applications.

8. Interpretation of Results in an Industry Context

For a cloud service provider, the DP-based 0/1 Knapsack solution directly translates to a data-driven admission-control policy: instead of accepting client applications on a first-come-first-served basis or by intuition, the provider can compute the exact combination of applications to host that maximizes revenue for the currently available memory capacity. As demonstrated by the sample dataset, this can mean rejecting or deferring a single high-revenue application (App5) in favor of hosting several other applications together, because the combined revenue and memory efficiency of the group is higher than that of the single high-value item alone.

This approach is scalable to other resource-allocation problems faced by cloud providers, such as allocating CPU cores, bandwidth, or storage among competing workloads, virtual machine placement, and multi-tenant SLA-based scheduling, wherever resources are limited and allocation decisions are effectively binary (accept/reject) rather than continuously divisible.

9. Conclusion

The 0/1 Knapsack Problem provides an effective and mathematically guaranteed optimal model for allocating limited server memory among competing client applications to maximize revenue. Dynamic Programming is the appropriate technique for this problem because it exploits both optimal substructure and overlapping subproblems to compute the true optimum in pseudo-polynomial $O(n \times W)$ time, in contrast to a greedy strategy, which is computationally cheaper but not guaranteed to produce an optimal allocation for the 0/1 (indivisible) case. The implementation, complexity analysis, and sample results confirm that this approach is both theoretically sound and practically deployable for real-world cloud resource-allocation decisions.